



Kubernetes 生态与 燧原 GPU 集成实践

基于 Operator / DRA / CDI 的 GPU 云原生
管理

燧原科技
2026

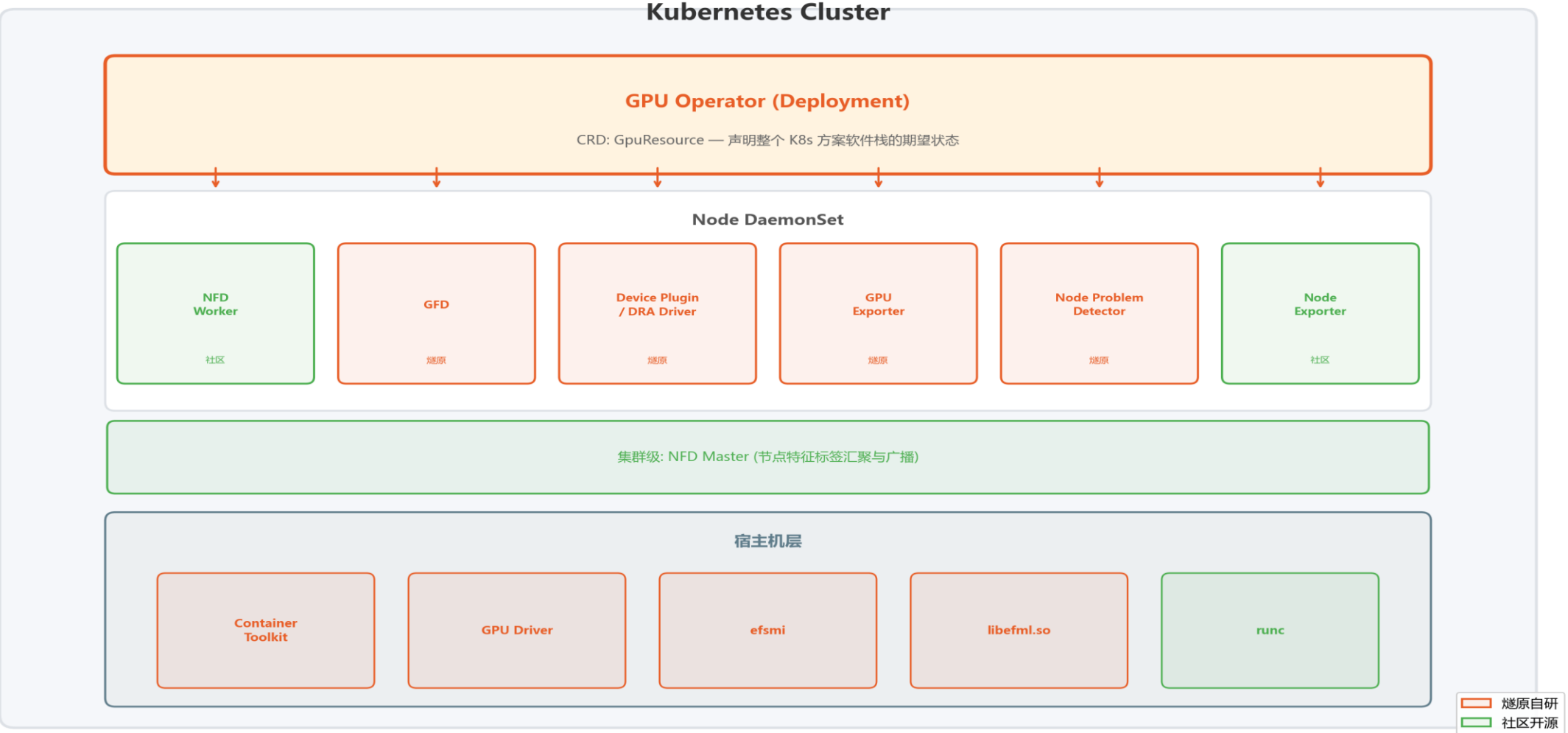
目录

- 1. 背景与挑战 — AI 基础设施对 K8s 异构设备管理的需求
- 2. 整体架构 — Operator 统一管控的 K8s 集成方案全景
- 3. 关键实践 — 统一管理 / 设备发现 / 资源调度 / 容器接入 / 可观测
- 4. 端到端 workflow — 从提交 Pod 到 GPU 推理的完整自动化链路
- 5. 实践总结 — 关键指标与效果
- 6. 全栈架构 — GPU 管理层与推理栈的衔接关系
- 7. CNCF 云原生推理栈 — 四层架构详解
- 8. LLM 推理工作计划 — 六个方向的接入与优化
- 9. Inference Gateway 深度 — K8s 推理网关的标准路由层
- 10. 展望 — 标准化演进与生态合作

1 背景与挑战

- 大模型训练与推理推动加速卡集群从数十卡扩展到数千卡
- Kubernetes 成为 AI 基础设施的事实标准
- 原生 K8s 对异构加速卡支持存在四类问题：
 - 设备不可见 — 调度器无法感知型号、显存、芯片系列
 - 资源难管理 — 缺少标准化的发现、注册、分配、回收机制
 - 运维复杂 — 驱动、运行时、监控需要大量人工操作
 - 生态碎片化 — 不同厂商方案各异，缺乏统一的云原生集成路径
- 燧原 K8s 集成方案基于 K8s 生态实现 GPU 从硬件到集群的全生命周期管理

2 整体架构



3.1 Operator 统一管理

- K8s 方案软件栈包含 7+ 组件
 - NFD、GFD、Device Plugin/DRA Driver、GPU Exporter
 - NPD、Node Exporter、Container Toolkit
- CRD 声明式管理：
 - apiVersion: enflame.com/v1
 - kind: GpuResource
 - spec: nfd/gfd/devicePlugin/gpuExporter/containerToolkit
- 效果：
 - 一键部署、滚动升级、自动 Reconcile
 - 1 个 CR 管理 7+ 组件，运维复杂度显著降低

3.2 设备发现自动化

- 硬件属性到 K8s 节点标签的全自动管道：
 - GPU 硬件 → efsmi / libefml → GFD → 标签文件 → NFD → Node Labels
- 三级后端自动回退，兼容不同驱动版本：
 - 1. efsmi -q --json-format (JSON, 推荐)
 - 2. efsmi -q (文本解析, 兼容旧版)
 - 3. libefml.so (CGO 绑定, 最终回退)
- 精准调度示例：
 - nodeSelector: enflame.com/gpu.model: S60
 - enflame.com/gpu.family: XXX / gpu.memory: 46080
- 标签每 60 秒刷新，驱动升级对上层工作流无感知

3.3.1 设备资源管理演进 — Device Plugin

- 方案一：Device Plugin (K8s $\geq$ 1.9)
- 通过 kubelet Device Plugin API 注册 GPU 为扩展资源：
 - `resources.limits: enflame.com/gpu: 2`
- 优势：简单可靠、生态成熟
- 局限：只能按数量分配整卡，无法指定型号或属性；无设备切分能力

3.3.2 设备资源管理演进 — DRA Driver 能力

- 方案二：DRA Driver (K8s >= 1.34)
- 核心概念：
 - DeviceClass — 集群级设备类型声明
 - ResourceClaim — 工作负载级设备请求 + CEL Selector
 - DRS (Dynamic Resource Slicing) — 整卡切分为逻辑切片
- Device Plugin vs DRA Driver：
 - 资源声明: limits 按数量 → ResourceClaim + CEL
 - 设备选择: 仅数量 → 按属性 (型号/显存/profile)
 - 设备切分: 整卡 → DRS 静态切片
 - 多设备组合: 不支持 → 支持
 - 设备注入: 环境变量 → CDI
- 燧原首批完成 DRA 适配，Operator 自动切换

3.3.3 设备资源管理演进 — DRA 使用示例（三步完成）

- Step 1: DeviceClass (Operator 自动部署)
 - kind: DeviceClass, metadata.name: enflame.com
- Step 2: ResourceClaim + CEL 精确选卡
 - deviceClassName: enflame.com
 - CEL: device.attributes['enflame.com'].model == 'S60'
- Step 3: Pod 引用 Claim → 调度器 + DRA Driver 自动分配 + CDI 注入
 - resourceClaims: [{name: gpu, resourceClaimName: my-gpu-claim}]
 - containers.resources.claims: [{name: gpu}]

3.3.4 设备资源管理演进 — DRS 静态切片

- 场景：一张 S60 整卡显存充足，多个推理任务共享
- CEL 按 profile 选切片：
 - `device.attributes['enflame.com'].profile == '1g.11gb'`
- 典型 Profile：
 - 1g.11gb — 4 切片, 1/4 显存, 小模型推理、开发调试
 - 2g.22gb — 2 切片, 1/2 显存, 中等模型推理
 - full — 1 切片, 完整显存, 训练 / 大模型推理
- 节点启动时划分，调度器按切片粒度分配
- 运行时隔离由驱动与 Container Toolkit 协同保证

3.4.1 容器透明接入 — Container Toolkit

- OCI runtime wrapper 拦截容器启动流程：
 - 1. 解析容器环境变量 / CDI Spec
 - 2. 修改 OCI runtime spec
 - 3. 注入 /dev/gpu*、libefml.so、驱动库
 - 4. 调用底层 runc 启动容器
- 两种接入模式：
 - Legacy: 环境变量 ENFLAME_VISIBLE_DEVICES (Device Plugin 路径)
 - CDI: CDI Spec + devices 字段 (DRA 路径, 推荐)
- 兼容 Docker / containerd / CRI-O, 与运行时解耦

3.4.2 容器透明接入 — CDI 机制

- CDI (Container Device Interface) — CNCF 标准设备注入规范
- 取代厂商私有环境变量方案
- CDI Spec: `/etc/cdi/enflame.com-gpu.yaml`
 - 声明 `deviceNodes`、`mounts`、`env`、`hooks`
- 优势：
 - 标准化 — K8s DRA、Podman、CRI-O 原生支持
 - 声明式 — Spec 描述注入内容，运行时负责执行
 - 可组合 — 多设备 (GPU + RDMA) 组合注入
 - 动态生成 — `enflame-ctl generate` 按硬件产出 Spec

3.5 全栈可观测

- 三层 Exporter 从芯片到集群：
 - 加速卡层: GPU Exporter — 利用率/显存/温度/功耗/ECC/PCIe
 - 系统层: Node Exporter — CPU/内存/磁盘/网络
 - 健康层: NPD — 内核错误/硬件异常/驱动故障
- 生态集成：
 - GPU 硬件 → libefml → GPU Exporter → Prometheus → Grafana
 - AlertManager → 告警
- 采集频率 15 秒，CPU 开销 < 1%
- Pod 级使用量与工作负载标签关联

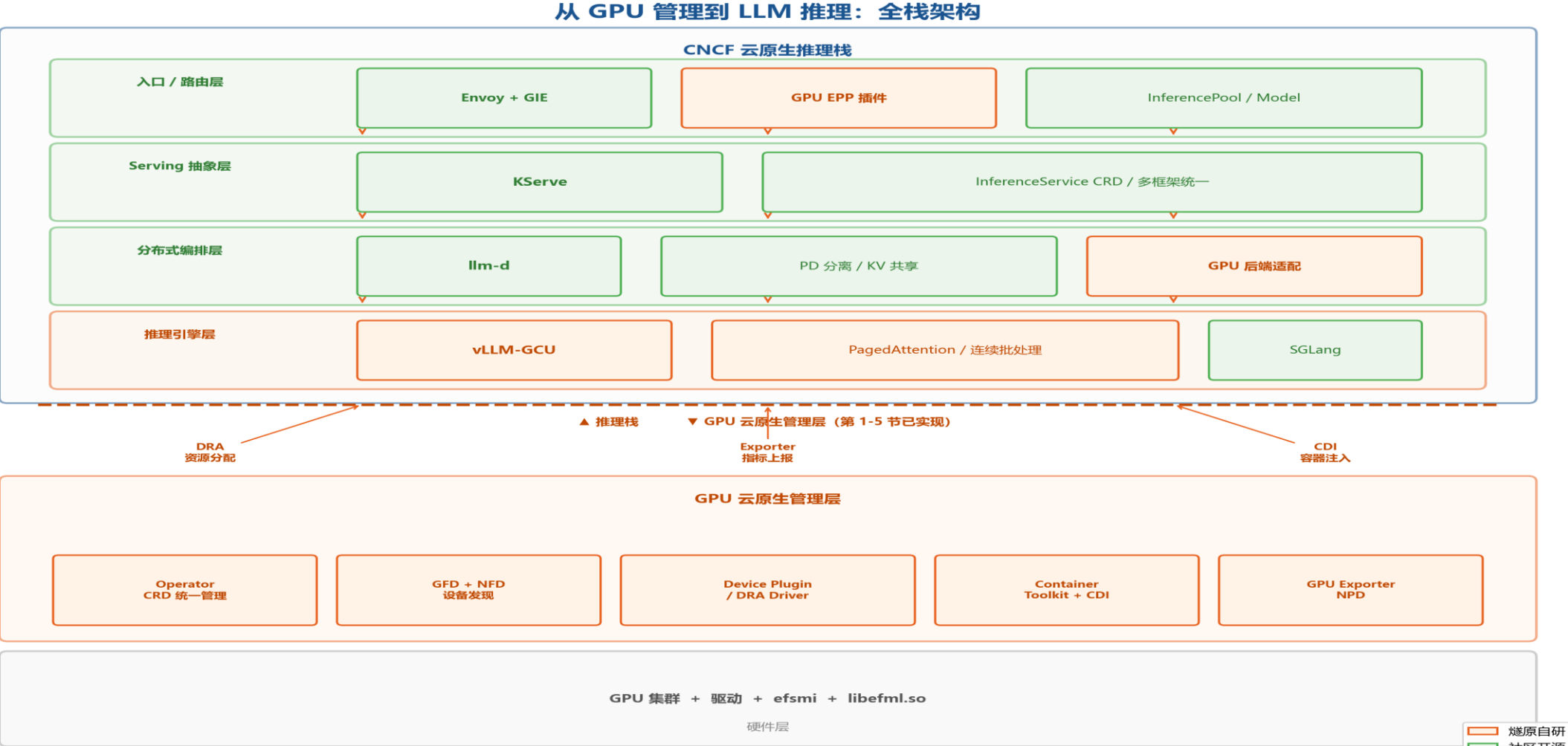
4 端到端 workflow

- 1. 用户提交 Pod (声明 GPU 资源需求)
 - 2. K8s Scheduler — 读取 GFD 节点标签 → 匹配 → 选择最优节点
 - 3. Kubelet + Device Plugin / DRA Driver — 分配设备 + 生成配置
 - 4. Container Toolkit — 注入 /dev/gpu* + 驱动库
 - 5. 容器启动, 应用使用 GPU 执行推理 / 训练
 - 6. 持续运行: GPU Exporter → Prometheus / NPD → K8s API / GFD → 标签刷新
-
- 用户视角: 提交 Pod, 其余自动化

5 实践总结

- 统一管理: Operator + CRD → 1 CR 管理 7+ 组件
 - 设备发现: GFD + NFD + 三级后端 → 60 秒刷新, 零人工
 - 资源调度: Device Plugin + DRA 双轨 → K8s 1.9 ~ 1.34+ 平滑过渡
 - 容器接入: Container Toolkit + CDI → 应用无需修改
 - 可观测性: 三层 Exporter + Prometheus → 秒级采集
-
- 燧原 GPU 在 K8s 中实现了与主流 GPU 同级的云原生管理能力
 - 使用标准 K8s 工作流, 无需关心底层硬件差异

6 从 GPU 管理到 LLM 推理：全栈架构



7 CNCF 云原生推理栈（详解）

- 大模型推理 → 分布式云原生推理：高并发/长上下文/KV cache/SLO
- CNCF 四层标准栈：
 - 入口/路由: Envoy + GIE — 模型感知路由, KV 打分, 多租户 (LLM 时代的 Ingress)
 - Serving: KServe — InferenceService CRD, 多框架统一 (推理的 Deployment)
 - 编排: llm-d — Prefill/Decode 分离, 跨 Pod KV 共享 (vLLM 分布式调度)
 - 引擎: vLLM / SGLang — PagedAttention, 连续批处理 (计算核心)
- GIE 与 llm-d 的分工（互补不替代）：
 - GIE 位于 Pod 之前，决定请求路由到哪个后端 Pod
 - llm-d 位于 Pod 之内 / 之间，决定请求在多 Pod 之间如何协同执行
- 燧原 K8s 集成方案目标：GPU 作为原生后端完整接入全部四层

8.1 LLM 推理工作计划 — 接入与路由

- 定位：基于 CNCF 四层栈，把燧原 GPU 接入为原生后端
- 接入 llm-d:
 - 对齐 vLLM 上游, 贡献 GPU 后端
 - Prefill/Decode 分离异构部署, 按负载独立伸缩
- Inference Gateway:
 - 集成 GIE, 导出 GPU 推理指标, 请求智能路由, P99 稳定
- 三级 KV Cache:
 - Prefix 感知路由 + LMCache (HBM → 主机 → NVMe)
 - 长上下文吞吐提升, 显存占用降低

8.2 LLM 推理工作计划 — 调度弹性与可观测

- 拓扑感知调度:
 - 多卡互联拓扑 → DRA 属性; TP/EP/PP 自动亲和 + Gang 调度
- SLO 驱动弹性:
 - TTFT/TPOT/队列深度驱动 HPA
 - Prefill / Decode 独立扩缩 + DRS 切片重组
- 推理级可观测:
 - GPU Exporter 扩展 TTFT/TPOT/KV 命中率/token 吞吐
 - 接入 Grafana LLM Dashboard
- 最终形态: 声明模型 + SLO, 调度 / 路由 / 扩缩 / 监控链路自动化

9.1 Inference Gateway 深度 — 背景与核心路由

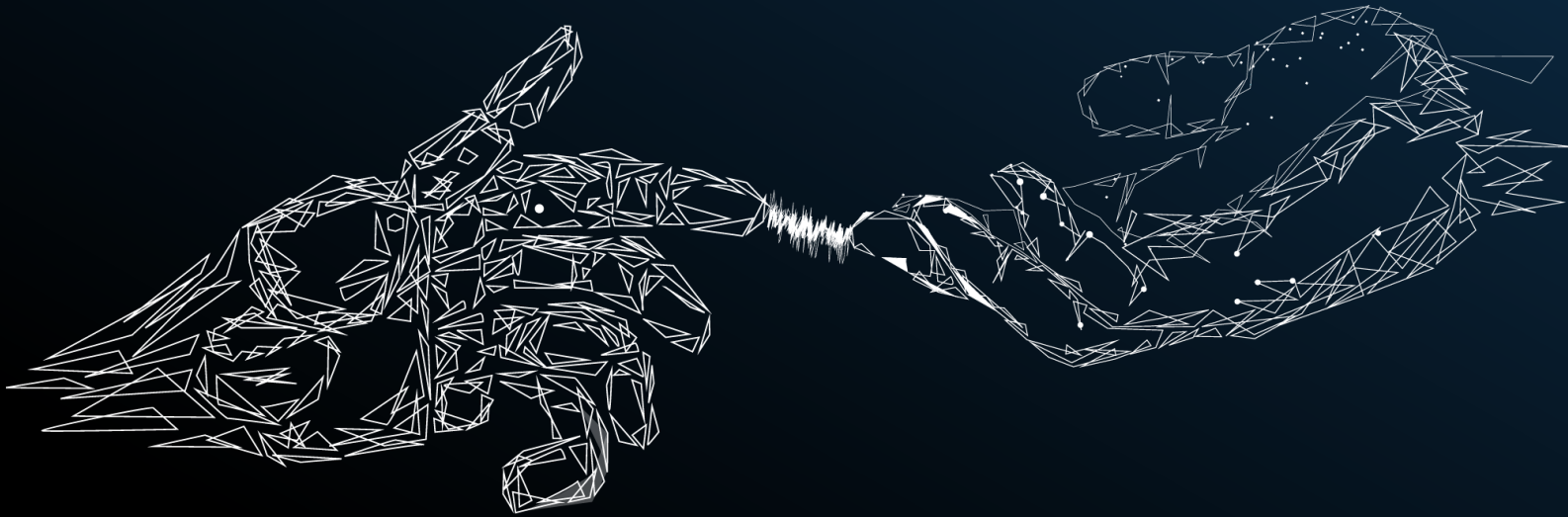
- GIE (Gateway API Inference Extension) — CNCF LLM 推理标准路由层
- 核心 CRD:
 - InferencePool — 推理后端组, DRA 分配 GPU
 - InferenceModel — 模型/版本/灰度/多租户
 - EPP (Endpoint Picker) — gRPC 插件, Gateway 调用选 Pod
- 燧原 K8s 集成方案工作项 — 路由与资源:
 - GPU EPP 插件 — 队列深度/KV命中率/显存余量打分, 接入 GIE gRPC
 - InferencePool x DRA — Pool 声明 GPU 型号 + DRS profile, 网关解耦
 - Prefix Cache 路由 — 相同前缀落同一节点, 提升 KV 命中
 - PD 分离路由 — Prefill/Decode 独立 InferencePool, 按阶段分发

9.2 Inference Gateway 深度 — 治理与生态

- 燧原 K8s 集成方案工作项 — 治理与生态：
 - SLO & 公平性 — 租户 priority/fairness + DRS 切片隔离，多租户 SLO 保证
 - 多 Gateway 兼容 — 对齐 Envoy / kgateway / Istio Ambient，跨网关一致
 - 灰度 & A/B — InferenceModel 多版本路由 + GPU Exporter 灰度指标，上线安全
- 定位：
 - 燧原 GPU 作为 K8s 推理网关的标准后端
 - 流量治理、灰度上线、多租户 SLO 全部对齐 K8s 控制面

10 展望

- DRA GA 与能力扩展
 - 更丰富的 CEL 属性、跨节点 ResourceClaim、Partitionable Devices
- Inference Gateway 标准化
 - GIE beta → GA, 多租户与 SLO 行为标准化
- CDI 生态收敛
 - containerd / CRI-O 原生 CDI 支持推进到 GA
- 开放生态合作
 - 与 CNCF、上游社区、框架厂商长期协作



■ Thank You ■